

Editing Entries

Now we'll make a page so users can edit the entries they've added.

The `edit_entry` URL

The URL for the page needs to pass the ID of the entry to be edited. Here's *learning_logs/urls.py*:

```
urls.py --snip--
urlpatterns = [
    --snip--
    # Page for editing an entry.
    path('edit_entry/<int:entry_id>/', views.edit_entry, name='edit_entry'),
]
```

This URL pattern matches URLs like *http://localhost:8000/edit_entry/id/*. Here the value of *id* is assigned to the parameter *entry_id*. Django sends requests that match this format to the view function `edit_entry()`.

The `edit_entry()` View Function

When the `edit_entry` page receives a GET request, the `edit_entry()` function returns a form for editing the entry. When the page receives a POST request with revised entry text, it saves the modified text into the database:

```
views.py from django.shortcuts import render, redirect

from .models import Topic, Entry
from .forms import TopicForm, EntryForm
--snip--

def edit_entry(request, entry_id):
    """Edit an existing entry."""
    ❶ entry = Entry.objects.get(id=entry_id)
    topic = entry.topic

    if request.method != 'POST':
        # Initial request; pre-fill form with the current entry.
        ❷ form = EntryForm(instance=entry)
    else:
        # POST data submitted; process data.
        ❸ form = EntryForm(instance=entry, data=request.POST)
        if form.is_valid():
            ❹ form.save()
            ❺ return redirect('learning_logs:topic', topic_id=topic.id)

    context = {'entry': entry, 'topic': topic, 'form': form}
    return render(request, 'learning_logs/edit_entry.html', context)
```

We first import the `Entry` model. We then get the entry object that the user wants to edit ❶ and the topic associated with this entry. In the if